6.1 A fresh two-page baseline

In this video we'll start a new Campus Supply site for class 6. We already know how PHP passes data to Twig. Today we ask a new question: should one class both choose a page and generate it?

Chapter 22 introduces an improved structure for object oriented PHP applications, utilizing a front controller that determines what to show and separate display controllers that determine how to show it.

First, we'll build a working baseline so we can observe that our later refactoring preserves its behavior.

Create the entry script

Create `public/tutorials/class6/index.php`:

```php
<?php
declare(strict_types=1);

use App\Tutorials\Class6\Application;

require_once __DIR__ . '/../../../vendor/autoload.php';

$app = new Application();
$app->run();
```

The three parent steps lead from class6 to tutorials, public, and the repository root. Composer already maps App classes to src. This entry script will remain unchanged throughout the tutorials.

Create the baseline Application

Create `src/Tutorials/Class6/Application.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

use Twig\Environment;
use Twig\Loader\FilesystemLoader;

class Application
{
    private const PATH_TO_TEMPLATES = __DIR__ .
'/../../../templates/tutorials/class6';

    private Environment $twig;

    public function __construct()
    {
```

```
        $loader = new FilesystemLoader(self::PATH_TO_TEMPLATES);
        $this->twig = new Environment($loader, [
            'autoescape' => 'html',
            'strict_variables' => true,
        ]);
    }

    public function run(): void
    {
        $action = $_GET['action'] ?? 'home';
        if (!is_string($action)) {
            $action = 'home';
        }

        switch ($action) {
            case 'contact':
                $this->contact();
                break;
            case 'home':
            default:
                $this->home();
        }
    }

    private function home(): void
    {
        $template = 'home.html.twig';
        $args = [];
        echo $this->twig->render($template, $args);
    }

    private function contact(): void
    {
        $template = 'contact.html.twig';
        $args = [];
        echo $this->twig->render($template, $args);
    }
}
```

The template path also climbs three levels, this time from Class6 through Tutorials and src to the repository root. Unknown, missing, or non-string actions select Home. The switch selects known methods; a query value never becomes an arbitrary method or template filename.

Create the shared layout and views

Create `templates/tutorials/class6/base.html.twig`:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Campus Supply: {% block pageTitle %}{% endblock %}</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header>
        <p>Campus Supply</p>
        <nav aria-label="Main navigation">
            <a class="{% block homeLink %}{% endblock %}" href="index.php?
action=home">Home</a>
            <a class="{% block contactLink %}{% endblock %}" href="index.php?
action=contact">Contact</a>
        </nav>
    </header>
    <main>
        <h1>{{ block('pageTitle') }}</h1>
        {% block main %}{% endblock %}
    </main>
</body>
</html>
```

Create `templates/tutorials/class6/home.html.twig`:

```
{% extends 'base.html.twig' %}
{% block pageTitle %}Home{% endblock %}
{% block homeLink %}active{% endblock %}
{% block main %}
    <p>Welcome to Campus Supply. Find supplies for your next study session.</p>
{% endblock %}
```

Create `templates/tutorials/class6/contact.html.twig`:

```
{% extends 'base.html.twig' %}
{% block pageTitle %}Contact{% endblock %}
{% block contactLink %}active{% endblock %}
{% block main %}
    <p>Email supplies@example.com or visit Library 204.</p>
{% endblock %}
```

Create `public/tutorials/class6/style.css`:

```
body {
    max-width: 52rem;
    margin: 2rem auto;
    padding: 0 1rem;
    font-family: system-ui, sans-serif;
    line-height: 1.5;
```

```
}
nav { display: flex; gap: 1rem; flex-wrap: wrap; }
nav a { color: #174c87; }
nav a.active { font-weight: bold; text-decoration-thickness: 0.2em; }
dt { font-weight: bold; }
dd { margin-bottom: 1rem; }
```

Verify and identify responsibilities

Start the server as we have before. Visit `/tutorials/class6/index.php`, then follow Contact and Home. The heading, tab title, and active link should agree. Try `?action=unknown` and `?action[]=contact`; both show Home.

Pause and locate two jobs in Application: run chooses what happens, while home and contact generate a response using Twig. The views already separate HTML from PHP, but the PHP class still mixes routing and display responsibilities. Our next tutorial separates those responsibilities into different classes.